PRACTICAL LAB

Test SO101 arm movement in Isaac Sim

SO-ARM101 · Isaac Sim 5.1 / Isaac Lab 2.3 · Learner guide

In this lab you open the SO-ARM101 in an Isaac Sim window, make the arm swing forward and backward 10 times, and watch what every joint does while it moves. The test checks that the simulator, the SO101 task and the live connection all work. Then you use the joint data to find out **why** the arm doesn't follow its command exactly, and you finish by turning the program into a gripper open/close test.

```
Part A: Run              Part B: Monitor          Part C: Understand       Part D + Exercise
launch sim  -->          monitor window,   -->    stiffness & damping, --> change the movement,
connect, swing arm       snapshot, CSV plot       sweep tool               open/close the gripper
(Steps 1-3)              (Step 4)                 (read your results)      (Steps E1-E7)
```

Learning outcomes

By the end of this lab you can:

1. Launch Isaac Sim with an SO101 task and connect to it from a second terminal.
2. Send a program into the running sim and make the arm move without freezing the GUI.
3. Read a joint's **position**, **target**, **error** and **torque** in four ways: the monitor window, a snapshot, the per-swing summary and a CSV plot.
4. Explain how stiffness and damping drive a joint, and use the sweep tool to find motion settings the joint can follow.
5. Adapt the movement program to another joint (the gripper) and decide from the data whether a joint is **weak**, **slow** or **at a limit**.

Time

Part	Time
First launch of Isaac Sim after an install	5–10 minutes of shader compiling (only once)
Part A–D (run, monitor, understand, change)	about 45 minutes
Exercise: open and close the gripper	about 30 minutes

What you need

Item	Value
Isaac Sim / Isaac Lab Python	`C:\isaacsimlab\env_isaaclab\Scripts\python.exe`
Workspace (run every command from here)	`C:\SO101wClaudeIsaac`
Launcher script	`tools\launch_live_sim.ps1`
Movement program	`tools\arm_forward_back.py`
Joint monitor window	`tools\joint_monitor.py` (opened automatically by the movement program)

Item	Value
Joint log plotter	`tools\plot_joint_log.py`
Sweep tool (Part C)	`tools\sweep_joint.py`
Joint logs (CSV)	`logs\arm_forward_back_<date>_<time>.csv`
Connection helper	`tools\sim_exec.py` (talks to the sim on `127.0.0.1:8226`)
Editor (exercise)	VS Code, with the `code` command available in PowerShell

Clone into C:\SO101wClaudeIsaac

git clone https://github.com/MuammerBay/isaac_so_arm101.git

> **NOTE**
> You do **not** need to activate a Python environment. Every command calls the `env_isaaclab` Python by its full path, so it doesn't matter if another prompt such as conda `(base)` is active.

How it works: two terminals

- **Terminal 1** runs Isaac Sim with the SO101 environment. It stays open for the whole lab and prints the results of each run.
- **Terminal 2** sends programs into that running sim with `sim_exec.py`. The programs run *inside* Isaac Sim, not in Terminal 2.

```
Terminal 2                        Terminal 1 / Isaac Sim window
sim_exec.py  arm_forward_back.py  ---- 127.0.0.1:8226 ---->   runs the program inside the sim
             print_joints()                                    arm moves, monitor window updates
"Started: ..." returns at once    <----------------------       per-swing lines + CSV log
```

Part A: Run the arm movement test

Step 1: Start Isaac Sim with the SO101 arm (Terminal 1)

Open a PowerShell terminal and run:

```
cd C:\SO101wClaudeIsaac
.\tools\launch_live_sim.ps1 -Task Isaac-SO-ARM101-Reach-Play-v0 -NumEnvs 2
```

- Wait until the Isaac Sim window opens and shows the SO101 arms: one per environment, so two with `-NumEnvs 2`.
- The first launch after installing can show nothing for **5–10 minutes** while shaders compile. This is normal; it is not a hang.
- Leave this terminal and the Isaac Sim window open.

IF THE SCRIPT WON'T RUN

If PowerShell says **running scripts is disabled on this system**, either allow local scripts once:

`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

or start the launcher with a one-off bypass:

`powershell -ExecutionPolicy Bypass -File .\tools\launch_live_sim.ps1 -Task Isaac-SO-ARM101-Reach-Play-v0 -NumEnvs 2`

CHECKPOINT: TICK EACH BOX BEFORE YOU GO ON

☐ The Isaac Sim window is open and shows two SO101 arms.

Step 2: Check the connection (Terminal 2)

Open a **second** PowerShell terminal and run:

```
cd C:\SO101wClaudeIsaac
$py = "C:\isaacsimlab\env_isaaclab\Scripts\python.exe"
& $py tools\sim_exec.py --ping
```

Expected output:

```
Isaac Sim live, kit 107.3.3+production...
```

WATCH OUT

Keep the `&` at the start. `& $py ...` tells PowerShell to run the program stored in `$py`. Without the `&`, PowerShell shows `Unexpected token 'tools\sim_exec.py' in expression or statement`.

`$py` only exists in the terminal where you set it. If you open a new terminal, run the `$py = ...` line again.

CHECKPOINT: TICK EACH BOX BEFORE YOU GO ON

☐ `--ping` prints `Isaac Sim live, kit ...`.

Step 3: Run the arm movement test (Terminal 2)

```
& $py tools\sim_exec.py tools\arm_forward_back.py
```

Terminal 2 prints this right away and returns:

```
Joint monitor open: env 0, joints shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper
Started: 10 x 2.0s swings of shoulder_lift (+/-0.6 rad), ~20s total
```

Now watch the Isaac Sim window. Every arm swings forward and backward 10 times, about 20 seconds in total, then goes back to its rest pose. An **SO101 Joint Monitor** window also opens inside Isaac Sim (you use it in Step 4). **Terminal 1** prints one line per finished swing, then a summary of every joint and the path of the saved log:

```
[arm_forward_back] cycle 1/10  shoulder_lift reached -0.26 .. +0.51 rad from default (commanded +/-0.6)
max error 0.58 rad  torque saturated 97% of steps
...
[arm_forward_back] cycle 10/10  shoulder_lift reached -0.26 .. +0.49 rad from default (commanded +/-0.6)
max error 0.57 rad  torque saturated 97% of steps
[arm_forward_back] done: 10 cycles
[arm_forward_back] joint summary over 600 steps (env 0):
  joint           min pos  max pos max |err| max |tau|  sat %
  shoulder_pan     -0.000   +0.001     0.001      0.92      0
  shoulder_lift    -0.265   +0.513     0.576      1.90     97
  elbow_flex       -0.036   +0.015     0.036      1.90      4
  wrist_flex       +1.460   +1.571     0.110      1.90      0
  wrist_roll       -0.005   +0.006     0.006      1.90      2
  gripper          -0.161   +0.003     0.161      2.50      9
[arm_forward_back] joint log saved: C:/SO101wClaudeIsaac/logs/arm_forward_back_20261008_002353.csv
```

You can run the Step 3 command again as many times as you like while the sim is open. Each run reopens the monitor window and writes a new CSV.

CHECKPOINT: TICK EACH BOX BEFORE YOU GO ON

- ☐ Both arms swung forward and backward 10 times, then returned to rest.
- ☐ Terminal 1 printed 10 `cycle` lines, a joint summary and a `joint log saved:` path.
- ☐ The **SO101 Joint Monitor** window opened in Isaac Sim.

Part B: Monitor the joints (Step 4)

Each joint has four values worth watching:

Value	Meaning	Unit
position	Where the joint actually is	rad
target	Where the motor is told to go (rest pose + the program's command)	rad
error	target − position. A large error means the joint can't keep up	rad
torque	How hard the motor pushes. Each motor has a maximum (1.9 N·m for the arm joints, 2.5 for the gripper). At the maximum the motor is **saturated**: it can't push harder, so the error grows.	N·m

You can watch them in Isaac Sim (4a) or from Python (4b–4d).

> READ MORE
> New to these numbers? Open `docs\so101-joint-table-explained.html` in a browser. It explains each column of the joint table in terms of the real SO101 hardware (the STS3215 servo's encoder, controller, motor and limits), with diagrams and a chart of a real test run.

4a. In Isaac Sim: the SO101 Joint Monitor window

`arm_forward_back.py` opens this window automatically. To open it on its own (for example to watch the arm at rest), run in Terminal 2:

```
& $py tools\sim_exec.py tools\joint_monitor.py
```

The window refreshes every frame and shows, for one arm (env 0):

- **A table with one row per joint**: position (rad and degrees), target, error, velocity and torque.
  - **error** turns **yellow** when the joint is more than 0.05 rad off target.
  - **torque** turns **red** and shows `SAT` when the motor is at its limit.
- **Position within limits**: a bar per joint. Empty = lower limit, full = upper limit. It turns **red** within 3 % of either end. `wrist_flex` is always red at rest; Part C explains why.
- **A plot** of one joint over the last ~10 s: green = position, grey = target. The gap between the lines is the tracking error. Choose the joint in the **plot joint** drop-down. It starts on the joint being swung.
- **The header** shows the episode time and who is moving the arm: `arm_forward_back running` or `zero agent`.

The window can be docked like any other Isaac Sim panel. To close it, click its **X**, or run `& $py tools\sim_exec.py -c "close_joint_monitor()"`. To watch a different arm or change the colour thresholds, edit the settings at the top of `tools\joint_monitor.py` (`ENV_ID`, `PLOT_SAMPLES`, `SAT_FRACTION`, `ERR_WARN_RAD`).

> **WHY NOT THE PROPERTY PANEL?**
>
> If you select a joint prim such as `/World/envs/env_0/Robot/joints/shoulder_lift`, the Property panel shows `state:angular:physics:position` and `drive:angular:physics:targetPosition`. Isaac Lab runs physics through Fabric and never writes these back to USD, so they always read `0`.
>
> The monitor window reads the same data the RL environment uses (`robot.data.joint_pos`, `joint_pos_target`, `joint_vel`, `applied_torque`), so it shows the real values.

4b. From Python: a snapshot of all joints (Terminal 2)

While the sim is open, run this at any time, during or after a swing:

```
& $py tools\sim_exec.py -c "print_joints()"

env 0  step 160/600  (swinging)
joint           pos rad     deg   target   error     vel  torque  limits
shoulder_pan     +0.000    +0.0   +0.000  -0.000   -0.00   -0.02  [-1.73, +1.73]
shoulder_lift    +0.064    +3.7   -0.520  -0.584   -1.42   -1.90  [-1.57, +1.57]  SAT
elbow_flex       -0.023    -1.3   +0.000  +0.023   +0.05   +0.64  [-1.52, +1.52]
wrist_flex       +1.565   +89.7   +1.570  +0.005   +0.01   +0.03  [-1.49, +1.49]
wrist_roll       -0.000    -0.0   +0.000  +0.000   +0.00   -0.00  [-2.46, +2.56]
gripper          -0.020    -1.1   +0.000  +0.020   +0.06   -0.00  [-0.08, +1.65]
```

Use `print_joints(1)` for the second arm. `print_joints()` is defined when `arm_forward_back.py` runs, so run Step 3 at least once first.

Other values you can read the same way:

Command (after `& $py tools\sim_exec.py -c`)	Shows
`"print(state['k'], total_steps)"`	Swing progress in steps (600 = done)
`"print(len(joint_log), joint_log[-1])"`	Number of logged steps and the newest row
`"print(robot.data.joint_pos)"`	Positions of every joint of every arm (one row per arm)
`"print(robot.data.applied_torque)"`	Torques of every joint of every arm

> **WATCH OUT**
>
> Inside `-c "..."`, use **single quotes** for Python strings (`state['k']`). PowerShell strips double quotes inside the command.

4c. From Python: the per-swing summary (Terminal 1)

The lines Terminal 1 printed in Step 3 summarise each swing:

- **reached A .. B rad from default**: the furthest the joint actually got back (A) and forward (B), compared with the commanded `+/-AMPLITUDE_RAD`.
- **max error**: the largest gap between target and position in that swing.
- **torque saturated N% of steps**: how often the motor was at its limit.

At the end, the joint summary repeats this for **every** joint over the whole run. Joints that weren't commanded to move should show small `max pos` ranges. Large values there mean the swing is shaking the rest of the arm.

Record your results

Copy the `shoulder_lift` values from **your** Terminal 1 into the yellow column. The reference column is from a reference run.

shoulder_lift	Your run	Reference run
Commanded range		−0.6 .. +0.6 rad
Reached range (cycle 10)		−0.26 .. +0.49 rad
Largest error (max |err|)		0.576 rad
Torque saturated (sat %)		97 %

4d. From Python: the CSV log and plots

Every step of env 0 is saved to `logs\arm_forward_back_<date>_<time>.csv` with columns `step`, `t`, then `pos_<joint>`, `target_<joint>`, `vel_<joint>` and `torque_<joint>` for all six joints. Open it in Excel, or plot it (the sim doesn't need to be running):

```
& $py tools\plot_joint_log.py                    # newest log, opens a window
& $py tools\plot_joint_log.py --out joints.png   # save as an image instead
```

You get one row per joint: position vs target (left) and torque (right).

CHECKPOINT: TICK EACH BOX BEFORE YOU GO ON

- ☐ You found the `shoulder_lift` row in the monitor window and saw its torque cell turn red (`SAT`).
- ☐ `print_joints()` printed a snapshot in Terminal 2.
- ☐ You filled in the **Record your results** table.
- ☐ `plot_joint_log.py` showed position vs target and torque for all six joints.

Part C: Understand what you see

Your results show that `shoulder_lift` does not follow its command. This part explains why, using the way the sim drives each joint.

Stiffness and damping: how the sim drives a joint

Each joint's motor is driven by a **PD controller**. "P" (proportional) is the stiffness part and "D" (derivative) is the damping part. Every physics step it decides how hard the motor pushes:

```
torque = stiffness × (target − position)  +  damping × (0 − speed)
         └─ spring: pulls toward target ─┘    └─ brake: resists motion ─┘
```

The result is then capped at the motor's maximum: 1.9 N·m for the arm joints, 2.5 N·m for the gripper. When the formula asks for more than the cap, the monitor shows `SAT`. The values for each joint are in `isaac_so_arm101\src\isaac_so_arm101\robots\trs_so101\so_arm101.py`:

Joint	Stiffness (N·m per rad)	Damping (N·m per rad/s)
`shoulder_pan`	200	80
`shoulder_lift`	170	65
`elbow_flex`	120	45
`wrist_flex`	80	30
`wrist_roll`	50	20
`gripper`	60	20

Stiffness is the spring. It pulls the joint toward its target, harder the further away it is. With stiffness 170, being 0.01 rad (0.6°) off target gives a 1.7 N·m pull. To hold a load, the joint has to sit slightly off target so the spring can push back: the offset is load ÷ stiffness.

Higher stiffness	Lower stiffness
Follows the target tightly, sags less under load	Softer, sags more, follows loosely
Reaches the torque cap with a smaller error	Gentler, but slower to correct errors
With too little damping, it overshoots and oscillates	Can feel floppy

Damping is the brake. It pushes against motion in proportion to speed. It stops the spring from overshooting, but it also fights every motion, including the motion you asked for.

Higher damping	Lower damping
Smooth, no overshoot	Faster response
Slow and sluggish: lags behind the target	Can overshoot and oscillate
Uses up motor torque just to move	More of the torque goes into following the target

How the two work together

- **Response delay ≈ damping ÷ stiffness.** Gripper: 20 ÷ 60 ≈ 0.33 s, the lag you will see in the exercise. Shoulder: 65 ÷ 170 ≈ 0.38 s.
- **There is a "just right" damping**, called *critical damping*: the fastest response without overshoot. It depends on stiffness and on how heavy the moving part is: about 2 × √(stiffness × inertia). For the shoulder it is roughly 3–5. Below it the joint overshoots, above it the joint gets sluggish. The shoulder's 65 is far above it.
- **Damping can use up the whole motor.** The damping part alone, damping × speed, can exceed the cap. For the shoulder that happens at 1.9 ÷ 65 = 0.03 rad/s, which is why it shows `SAT` whenever it moves.

What this looks like on the shoulder, from the sweep tool (below):

Shoulder damping	At rest	Swing 0.6 rad / 2 s	Swing 0.3 rad / 2 s
65 (config)	0.13 rad off target, `SAT` 100 %	error 0.57, `SAT` 98 %	error 0.35, `SAT` 100 %
10	0.02 rad off, no `SAT`	error 0.54, `SAT` 63 %	error 0.06, no `SAT`
5	0.01 rad off, no `SAT`	**overshoot**: error 1.10, `SAT` 82 %	error 0.03, no `SAT`

Too much damping and the joint can't move freely. Too little and it overshoots.

> **ON THE REAL ROBOT**
> On the real SO101, the STS3215 servo runs the same kind of controller inside itself: its `P_Coefficient` plays the role of stiffness and its `D_Coefficient` the role of damping. They use the servo's own units, so the numbers aren't directly comparable. Tuning the sim's stiffness and damping until its joint logs match logs recorded on the real arm is how you make the sim behave like your robot.

What the default test shows

With the default settings, the monitoring shows that `shoulder_lift` **can't follow the command**:

- It is commanded ±0.6 rad but only reaches about **–0.26 to +0.49 rad**. Gravity helps it forward and holds it back, so the swing is lopsided.
- Its torque is at the **1.9 N·m limit 97 % of the time** (red `SAT` in the window).
- Even at rest it sits at about +0.13 rad instead of 0, with `SAT`.
- `wrist_flex` rests at 1.57 rad, slightly beyond its soft limit (±1.49), so its limit bar is always full and red. That comes from the robot's default pose, not from this test (see below).

The motor isn't too weak. Holding the arm in its rest pose against gravity takes only **0.46 N·m**, 24 % of the motor's 1.9 N·m. The cause is the shoulder drive's **damping** setting in the robot config (`so_arm101.py`: stiffness 170, damping 65).

The damping term brakes the joint in proportion to its speed. With damping 65, the damping term alone reaches the 1.9 N·m limit as soon as the joint moves faster than 1.9 ÷ 65 = **0.03 rad/s** (under 2° per second). So almost any motion saturates. Even at rest, PhysX reports a small leftover speed of about 0.25 rad/s on this joint. With damping 65 that becomes 16 N·m of braking, which leaves the joint stuck 0.13 rad off target.

Finding settings that work: the sweep tool

`tools\sweep_joint.py` tries every combination of amplitude and period in one go and prints a table: % of steps saturated, and largest error. The first cycle of each trial is ignored, because the joint is still settling. Run it like the other programs (it takes about 3 minutes of sim time):

```
& $py tools\sim_exec.py tools\sweep_joint.py
```

Results for `shoulder_lift` with the **current robot config (damping 65)**. Each cell is `SAT %` / largest error (rad):

amplitude \ period	2 s	3 s	4 s	6 s
0 (hold still)	100 % / 0.13			
0.1 rad	100 % / 0.20	100 % / 0.19	100 % / 0.18	100 % / 0.16
0.2 rad	100 % / 0.28	100 % / 0.25	100 % / 0.23	100 % / 0.20
0.3 rad	100 % / 0.35	100 % / 0.31	100 % / 0.28	100 % / 0.24
0.4 rad	100 % / 0.42	100 % / 0.37	98 % / 0.33	96 % / 0.27
0.6 rad	98 % / 0.57	97 % / 0.48	96 % / 0.42	96 % / 0.34

No amplitude and period combination clears `SAT` with the current config. Even holding still saturates.

The sweep tool can also try a different damping for the duration of the sweep only (`DAMPING_OVERRIDE` at the top of the file; the original value is restored afterwards). Results with **damping 10**, where ✓ = good (`SAT` ≤ 5 % and error ≤ 0.1 rad):

amplitude \ period	2 s	3 s	4 s	6 s
0 (hold still)	✓ 0 % / 0.02			
0.1 rad	✓ 0 % / 0.03	✓ 0 % / 0.03	✓ 0 % / 0.03	✓ 0 % / 0.03
0.2 rad	✓ 0 % / 0.05	✓ 0 % / 0.04	✓ 0 % / 0.04	✓ 0 % / 0.03
0.3 rad	✓ 0 % / 0.06	✓ 0 % / 0.05	✓ 0 % / 0.04	✓ 0 % / 0.04
0.4 rad	15 % / 0.09	✓ 0 % / 0.06	✓ 0 % / 0.05	✓ 0 % / 0.04
0.6 rad	63 % / 0.54	28 % / 0.10	9 % / 0.06	9 % / 0.05

With damping 10, good settings for `arm_forward_back.py` are:

Goal	AMPLITUDE_RAD	PERIOD_S
Fastest clean swing	`0.3`	`2.0`
Bigger swing	`0.4`	`3.0`
Biggest swing (`SAT` just over 5 %)	`0.6`	`4.0` or `6.0`

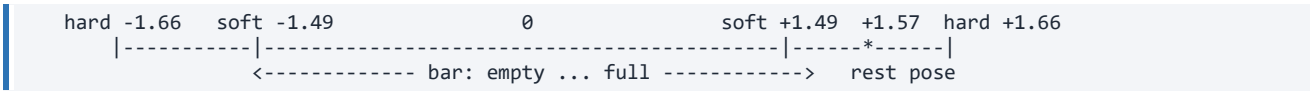
Damping 5 tracks even more closely at small amplitudes, but at 0.6 rad / 2 s the joint overshoots (error 1.10 rad), so damping 10 is the safer choice.

> **WATCH OUT**
>
> **Changing the damping is a robot-config decision, not a test setting.** The RL and imitation-learning tasks use the same robot config, so policies trained with damping 65 may behave differently with 10. Test it with the sweep first. If you change `so_arm101.py`, commit the change on its own and retrain.

Why the wrist_flex limit bar is always red

In the monitor window, `wrist_flex`'s **position within limits** bar is full and red even when nothing moves. The joint isn't in trouble. Its rest pose is slightly outside the range the bar is drawn against.

	wrist_flex
Rest pose (default position and target)	**+1.57 rad (90°)**
Soft limit, used by the bar and the `limits` column	±1.49 rad (85.5°)
Hard limit, the joint's real end of travel	±1.66 rad (95°)



Where the numbers come from, all in `so_arm101.py` and the robot's URDF:

- The **hard limit** (±1.66 rad) is set in the URDF.
- The **soft limit** is the hard limit shrunk to 90 %, by `soft_joint_pos_limit_factor=0.9`. Isaac Lab adds this safety margin for learning tasks.
- The **rest pose** is `"wrist_flex": 1.57`. It was chosen without allowing for that 90 % margin.

The bar is full because 1.57 is above the soft limit's upper end. It is red because the monitor colours any bar red within 3 % of either end.

Is it a problem?

- **Physically, no.** 1.57 rad is still 0.09 rad (about 5°) inside the hard limit, so the wrist isn't pressing against anything. Its torque (about +0.02 N·m) and error (about 0.001 rad) are both tiny.
- **It has one side effect, at every episode reset.** The Reach task randomises the joints to 0.5–1.5 × the rest pose, then clips them to the soft limits (`reset_joints_by_scale` in Isaac Lab). The wrist therefore starts each episode somewhere between 0.79 and 1.49 rad, never at its 1.57 target. In about half of all resets the random value is clipped to exactly 1.49. The motor then drives the wrist up to 1.57 in the first second. This causes the large `wrist_flex` errors you see at the start of a run, for example 0.445 rad in the gripper exercise's summary.

How it could be made green. Both options change the shared robot config, which the RL and imitation-learning tasks also use, so policies trained before the change may need retraining:

1. Move the rest pose inside the soft limit, for example `"wrist_flex": 1.45` (83°).
2. Use the full joint range: `soft_joint_pos_limit_factor=1.0`, as the SO-ARM100 config already does.

For this lab, the red bar can be ignored. Other joints are different: a red bar on them during a motion does mean the joint is near the end of its travel.

Check your understanding

Answer in the boxes. Answers are in Appendix C.

Q1. With damping 65, above what joint speed does the damping term alone saturate the shoulder motor? Show the calculation.

Your answer

Q2. Holding the arm still against gravity needs only 0.46 N·m. Why does the shoulder still show `SAT` at rest?

Your answer

Q3. Estimate how far behind its target the gripper will lag, in seconds. Which two settings did you use?

Your answer

Q4. Why is the `wrist_flex` limit bar always red, and does it harm the wrist?

Your answer

Q5. Why should you not simply change the shoulder damping to 10 in `so_arm101.py` and carry on?

Your answer

Part D: Change the movement

Edit the settings at the top of `tools\arm_forward_back.py`, save, then run Step 3 again:

Setting	Default	Meaning
`CYCLES`	`10`	Number of forward-and-back swings
`PERIOD_S`	`2.0`	Seconds for one swing
`AMPLITUDE_RAD`	`0.6`	How far the joint moves from its rest position, in radians
`JOINT`	`"shoulder_lift"`	Joint to move: `shoulder_pan`, `shoulder_lift`, `elbow_flex`, `wrist_flex`, `wrist_roll` or `gripper`

For example, `JOINT = "shoulder_pan"` swings the arm left and right instead. Keep `AMPLITUDE_RAD` inside the joint's limits (about ±1.5 rad for most joints).

Monitoring settings, in the same file:

Setting	Default	Meaning
`SHOW_MONITOR_WINDOW`	`True`	Open the SO101 Joint Monitor window in Isaac Sim. `False` = terminal output and CSV only
`SAVE_CSV`	`True`	Save the joint log to `logs\` when the run ends
`SATURATION_FRACTION`	`0.98`	Torque at or above this fraction of the motor's maximum counts as saturated

Try it

Set `JOINT = "shoulder_pan"`, run Step 3, and note the `shoulder_pan` values from the joint summary. Then put the setting back.

shoulder_pan	Your run
Reached range	
Largest error	
Torque saturated	
Does it follow the command better or worse than shoulder_lift? Why?	

Why the program doesn't freeze the GUI

The launcher runs the `zero_agent` script, which sends a "do nothing" command to every arm on every frame. `arm_forward_back.py` temporarily puts its forward/back command in place of that "do nothing" command while the existing loop keeps running, so the GUI stays responsive. For the duration of the test it also turns off the 12-second episode reset. When the 10 swings are done, it restores both.

The joint monitoring works the same way. The program records the joints inside its replacement step. The monitor window updates from Isaac Sim's per-frame update event. Neither one runs a loop of its own.

Step 5: Stop the test

Close the Isaac Sim window, or press `Ctrl+C` in Terminal 1. Start it again with Step 1 when you begin the exercise.

Exercise: Open and close the gripper

In this exercise you turn the arm-swing program into a gripper program, run it, and monitor the gripper the same way you monitored the shoulder in Step 4. It takes about 30 minutes.

You will practise:

- Copying a working program and changing only what you need.
- Writing a motion that suits a joint that only moves one way.
- Reading the joint monitor to decide whether a joint is **weak**, **slow**, or **at a limit**.

What you need

Item	Path
Starting program (you copy it)	`tools\arm_forward_back.py`
Your new program (you create it)	`tools\gripper_open_close.py`
Joint monitor window	`tools\joint_monitor.py` (opens automatically, as before)
Log plotter	`tools\plot_joint_log.py` (plots the newest CSV in `logs\`)
Reference solution, to check your work	`tools\solutions\gripper_open_close.py`

Facts about the gripper joint, which you will need below:

Property	Value	Where it comes from
Joint name	`gripper`	Last row of the joint table
Closed / open	`0` rad = closed. Positive = open. At 1.1 rad the moving jaw has swung about 28 mm away from the fixed jaw.	Measured in the sim
Allowed range (soft limits)	−0.08 to +1.65 rad	`limits` column
Mechanical range (hard limits)	−0.17 to +1.75 rad	Robot model (URDF)
Motor maximum in the sim	2.5 N·m (the arm joints have 1.9)	Robot config
Real servo	STS3215, stalls at about 1.9 N·m and is rated for 0.5 N·m continuous	Datasheet
Controller	stiffness 60, damping 20	Robot config
Speed cap	1.5 rad/s	Robot config

Why the arm program can't be reused as-is

`arm_forward_back.py` moves the joint along a sine wave **around** the rest pose: 0 → +A → 0 → −A → 0. That suits the shoulder, which can move both ways. The gripper's rest pose (0) is already closed, so the −A half would push the jaw into its closed stop.

The gripper needs a wave that goes **only one way**, closed → open → closed:

```
target = A × 0.5 × (1 − cos(2π t / T))
```

Time in the cycle	0	T/4	T/2	3T/4	T
`0.5 × (1 − cos(...))`	0	0.5	1	0.5	0
Gripper	closed	half open	fully open (A)	half open	closed

The wave also starts and ends with zero speed, so the jaw doesn't jerk.

Step E1: Start the sim and look at the gripper at rest

1. Start the sim and check the connection, as in Step 1 and Step 2.
2. In Terminal 2, open the monitor window:

   ```
   & $py tools\sim_exec.py tools\joint_monitor.py
   ```

3. In the window's **plot joint** drop-down, choose `gripper`.
4. Look at the `gripper` row. At rest it should read: position ≈ 0, target 0, error ≈ 0, torque ≈ 0. The jaw is closed and the motor is doing nothing.
5. In the Isaac Sim viewport, click the gripper of one arm, then press **F** to zoom to it. Keep this view so you can watch the jaw move.

Step E2: Copy the program (Terminal 2)

```
Copy-Item tools\arm_forward_back.py tools\gripper_open_close.py
code tools\gripper_open_close.py
```

The second line opens the copy in VS Code. Make all the changes below in **`tools\gripper_open_close.py`**, not in the original.

> **WATCH OUT**
>
> Edit the file in VS Code. Don't use PowerShell's `Set-Content` to edit it: it saves the file with a byte-order mark (BOM), and the sim then refuses the code with `invalid non-printable character U+FEFF`.

Step E3: Make the five changes

Change 1: settings. Near the top, in section 1, change these four lines:

```
# before
CYCLES = 10
PERIOD_S = 2.0
AMPLITUDE_RAD = 0.6
JOINT = "shoulder_lift"

# after
CYCLES = 5            # 5 open/close cycles
PERIOD_S = 3.0        # 3 s per cycle, slow enough for the gripper to follow
AMPLITUDE_RAD = 1.2   # open to 1.2 rad (69°), inside the 1.65 rad soft limit
JOINT = "gripper"
```

Change 2: the program's name. Press **Ctrl+H** (Find and Replace), find `arm_forward_back`, replace with `gripper_open_close`, and click **Replace All**. It should report 7 replacements. This changes the labels in Terminal 1 (`[gripper_open_close] ...`) and the log file name (`logs\gripper_open_close_<time>.csv`), so your gripper runs don't mix with the arm runs.

Change 3: the motion. In section 7, inside `def step(actions):`, find:

```
    actions[:, joint_idx] = AMPLITUDE_RAD * math.sin(2 * math.pi * t / PERIOD_S) / scale
```

and replace it with:

```
    actions[:, joint_idx] = AMPLITUDE_RAD * 0.5 * (1 - math.cos(2 * math.pi * t / PERIOD_S)) / scale
```

Keep the `/ scale` at the end. It converts radians into action units, as explained in the comments above that line.

Change 4: the per-cycle summary text. In `print_cycle_summary`, find:

```
        f"{s['hi'] - default:+.2f} rad from default (commanded +/-{AMPLITUDE_RAD})  "
```

and replace it with:

```
        f"{s['hi'] - default:+.2f} rad from default (commanded 0 .. +{AMPLITUDE_RAD})  "
```

Change 5: the start message. On the last line of the file, find:

```
print(f"Started: {CYCLES} x {PERIOD_S}s swings of {JOINT} (+/-{AMPLITUDE_RAD} rad), ~{CYCLES *
PERIOD_S:.0f}s total")
```

and replace it with:

```
print(f"Started: {CYCLES} x {PERIOD_S}s open/close cycles of {JOINT} (0 .. +{AMPLITUDE_RAD} rad), ~{CYCLES
* PERIOD_S:.0f}s total")
```

Optionally, update the comments too: the first line of the file and the comment above the motion line still describe a swing.

Save the file (**Ctrl+S**), then check it before you run it:

```
& $py -m py_compile tools\gripper_open_close.py
code --diff tools\gripper_open_close.py tools\solutions\gripper_open_close.py
```

- The first command prints nothing if there is no syntax error.
- The second shows your file next to the solution. The solution has an updated description at the top and a few rewritten comments. Every line of actual code should match.

CHECKPOINT: TICK EACH BOX BEFORE YOU GO ON

☐ `py_compile` printed nothing.

☐ `code --diff` shows differences only in comments.

Step E4: Run it (Terminal 2)

```
& $py tools\sim_exec.py tools\gripper_open_close.py
```

Terminal 2 prints:

```
Joint monitor open: env 0, joints shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper
Started: 5 x 3.0s open/close cycles of gripper (0 .. +1.2 rad), ~15s total
```

In the viewport, the jaw opens and closes 5 times over about 15 seconds, then stays closed. The monitor window's plot switches to `gripper` by itself.

Step E5: Monitor the gripper

Use the same four tools as Step 4:

1. **Monitor window (4a).** Watch the `gripper` row and the plot. The green line (position) follows the grey line (target) with a delay. Check whether the torque cell ever turns red.
2. **Snapshot (4b).** While it moves, run this a few times:

```
& $py tools\sim_exec.py -c "print_joints()"
```

3. **Per-cycle summary (4c).** Terminal 1 prints one line per cycle, then the joint summary:

```
[gripper_open_close] cycle 1/5  gripper reached +0.00 .. +1.09 rad from default (commanded 0 .. +1.2)
max error 0.34 rad  torque saturated 0% of steps
[gripper_open_close] cycle 2/5  gripper reached +0.11 .. +1.09 rad from default (commanded 0 .. +1.2)
max error 0.33 rad  torque saturated 0% of steps
...
[gripper_open_close] done: 5 cycles
[gripper_open_close] joint summary over 450 steps (env 0):
  joint           min pos  max pos max |err| max |tau|  sat %
  shoulder_pan     +0.000   +0.000     0.000      0.01      0
  shoulder_lift    +0.010   +0.128     0.128      1.90    100
  elbow_flex       -0.006   +0.020     0.020      1.90      1
  wrist_flex       +1.125   +1.577     0.445      0.39      0
  wrist_roll       -0.000   +0.000     0.000      0.01      0
  gripper          +0.000   +1.091     0.336      0.00      0
[gripper_open_close] joint log saved: C:/SO101wClaudeIsaac/logs/gripper_open_close_<date>_<time>.csv
```

4. **Plot (4d).**

```
& $py tools\plot_joint_log.py
```

5. Look at the bottom row (`gripper`). Check the scale on each axis before you compare plots: a label such as `1e-5` above a plot means its values are tiny (0.00001 rad). For still joints, the wiggle you see is that small.

Step E6: Read your results

Fill in the yellow column from **your** run. The other columns show a reference gripper run and the arm-swing test for comparison.

Question	Your gripper run	Reference gripper	Shoulder (arm test)
Commanded range		0 .. +1.2 rad	−0.6 .. +0.6 rad
Reached range		+0.11 .. +1.09 rad	−0.26 .. +0.49 rad
Largest error		0.33 rad	0.58 rad
Torque at the limit		0 % of steps	97 % of steps
Largest speed		0.98 rad/s (cap 1.5)	1.5 rad/s (at the cap)
Verdict		**Slow, not weak**	**Not weak either: damping 65 makes it saturate**

How to read it

- **The gripper isn't weak.** Its torque stays near 0 and never shows `SAT`. The jaw is light (about 34 g) and has almost no load to fight.
- **It is slow because of its controller settings.** The drive's damping (20) is large compared with its stiffness (60). Damping brakes the joint in proportion to its speed, so the jaw follows the target with a delay of about damping ÷ stiffness = 20 ÷ 60 ≈ 0.33 s. The pushing and braking almost cancel, which is why the torque reads about 0.
- **It doesn't fully close between cycles.** At the end of each cycle the target is back at 0, but the jaw is still about 0.11 rad behind and still closing. That is the same 0.33 s delay.
- **Rows you can ignore.** The `wrist_flex` error (0.445) happens in the first second. The reset at the start of the run clips the wrist below its soft limit, and the motor then drives it back up to its 1.57 rad rest pose (see Part C, *Why the wrist_flex limit bar is always red*). `shoulder_lift` at 100 % `SAT` is the damping effect explained in *What the default test shows*. Neither is caused by the gripper.

Step E7: Experiments

For each experiment, write down your **prediction** first. Then change one setting in `tools\gripper_open_close.py`, save it, run Step E4 again and compare. Put the setting back before the next experiment. Reference results are in Appendix B: look at them only after you have written your own.

Experiment A: open and close 3× faster

Set `PERIOD_S = 1.0`.

Prediction: will the jaw still open and close fully? Will the motor saturate?

Result: reached range, largest error, largest speed, SAT %

Experiment B: try to open past the limit

Set `AMPLITUDE_RAD = 2.0` (the soft limit is 1.65 rad).

Prediction: where will the jaw stop? What will the torque do?

Result: furthest position reached, SAT %, largest error

Experiment C: does the gripper disturb the arm?

Keep the original settings. Look at the other rows of the joint summary and at their torque plots.

Prediction and result: which joints move or change torque while the gripper works, and by how much?

What this means on the real SO101

- **Units.** The real gripper servo reports steps (4096 per turn) like every other joint. LeRobot shows the gripper as 0–100 % open instead of radians.
- **Grip strength.** The sim gives the gripper 2.5 N·m, but the real STS3215 stalls at about 1.9 N·m and is rated for 0.5 N·m continuous. Holding an object closed is a continuous load, so the real gripper heats up if you squeeze hard for a long time. Experiment B shows the same situation in the sim: `SAT` while pressed against something.
- **Speed and delay.** The sim's 0.33 s delay comes from its stiffness and damping settings. The real servo has its own P/D settings, so its delay will differ. Comparing a sim log with a real log is how you tune the sim to match the real arm.

Appendix A: Troubleshooting

Running the test

Problem	Cause and fix
`Cannot reach Isaac Sim at 127.0.0.1:8226`	The sim is not running, is still loading, or was started some other way than with `launch_live_sim.ps1`. Do Step 1 and wait for the window.
`Unexpected token 'tools\sim_exec.py'`	The `&` before `$py` is missing. Use `& $py tools\sim_exec.py ...`.
`The term '...python.exe' is not recognized`	`$py` was not set in this terminal. Run the `$py = "..."` line from Step 2 again. Variables don't carry over between terminals.
`StopIteration` or `ModuleNotFoundError: isaaclab` when running the program	The sim was started without a task (no `-Task`). Restart Step 1 with `-Task Isaac-SO-ARM101-Reach-Play-v0`.
Window shows nothing for several minutes	The first launch is compiling shaders. Wait 5–10 minutes.
`running scripts is disabled on this system`	See the execution-policy note in Step 1.
`NameError: name 'print_joints' is not defined`	`print_joints()` is created by `arm_forward_back.py`. Run Step 3 once first. Restarting the sim clears it.
`NameError` on a name you didn't type, e.g. `name 'robot' is not defined` after `env.scene["robot"]`	PowerShell removed the double quotes inside `-c "..."`. Use single quotes inside: `env.scene['robot']`.
Joint Monitor window doesn't appear	It may be docked behind another panel or off-screen. Run `& $py tools\sim_exec.py tools\joint_monitor.py` to recreate it.

Problem	Cause and fix
Monitor values stop changing	The env was closed or an error occurred. Terminal 1 shows `[joint_monitor] stopped: ...`. Reopen the window.

Gripper exercise

Problem	Cause and fix
`SyntaxError` when running, or `py_compile` reports an error	A typo in one of the changes. Compare with the solution using `code --diff` (Step E3).
`ValueError: 'grippr' is not in list`	`JOINT` is misspelt. It must be exactly `"gripper"`.
Terminal 1 still prints `[arm_forward_back]`, or the CSV is named `arm_forward_back_...`	Change 2 wasn't done or wasn't saved.
The target goes below 0, or the jaw presses against its closed stop	Change 3 is missing: the motion line still uses `math.sin`.
`invalid non-printable character U+FEFF`	The file was saved with a BOM by a PowerShell command. Re-save it from VS Code with **UTF-8** encoding (bottom-right of the status bar), not **UTF-8 with BOM**.
`plot_joint_log.py` shows an arm run	It plots the newest CSV. Pass the gripper file explicitly: `& $py tools\plot_joint_log.py logs\gripper_open_close_<date>_<time>.csv`.

Appendix B: Reference results for Experiments A–C

Experiment A: `PERIOD_S = 1.0`

- Reached only +0.31 .. +0.85 rad: the jaw neither opens nor closes fully.
- Largest error 0.71 rad in the first cycle, then about 0.54.
- Speed pinned at **1.5 rad/s**, the sim's speed cap. The cycle needs up to 3.8 rad/s (1.2 × π ÷ 1.0).
- Torque at its 2.5 N·m limit 7–27 % of the time.

Now the gripper is limited by **speed**, and the motor starts to saturate while it tries to catch up.

Experiment B: `AMPLITUDE_RAD = 2.0`

- The jaw stops at **+1.75 rad**, the mechanical (hard) limit, not at the 1.65 rad soft limit. In Isaac Lab, soft limits are used by learning tasks and the monitor, but nothing stops a target from going past them.
- While the target is beyond 1.75, the motor keeps pushing the jaw into its stop: torque at 2.5 N·m (`SAT`) for about 35 % of the time, error up to 0.6 rad.
- On the real arm, `Max_Position_Limit` (set during calibration) would normally refuse such a goal. If it didn't, the servo would stall against the printed jaw stop and heat up.

Experiment C: effect on the arm

Hardly at all. The other joints' positions barely change (elbow at most 0.02 rad, mostly at the start of the run). The wrist's torque ripples by about 0.2 N·m in time with the gripper, because the wrist carries the moving jaw. Compare this with the arm test, where swinging the shoulder shook every joint.

Appendix C: Answers to Check your understanding

1. Above 1.9 ÷ 65 ≈ **0.03 rad/s** (under 2° per second). At that speed the damping term (damping × speed) alone reaches the 1.9 N·m cap.
2. PhysX reports a leftover speed of about 0.25 rad/s on the joint even at rest. With damping 65 that is about 16 N·m of braking, far above the 1.9 N·m cap, so the motor saturates and the joint stays about 0.13 rad off target. The problem is the damping, not the motor's strength.
3. About damping ÷ stiffness = 20 ÷ 60 ≈ **0.33 s**, using the gripper's damping (20) and stiffness (60).
4. Its rest pose (1.57 rad) is above the soft limit (±1.49 rad) that the bar is drawn against, and the bar turns red within 3 % of either end. It does not harm the wrist: 1.57 rad is still about 0.09 rad inside the hard limit (1.66 rad), and torque and error are tiny. Its only side effect is a large wrist error in the first second after each reset.
5. The RL and imitation-learning tasks share the same robot config, so policies trained with damping 65 may behave differently with 10. Test the change with the sweep tool first, commit it on its own, and retrain.